

BLOCK 1/3 — TRADE_LIFECYCLE 정본 v0.1.5 (통합본) (0~5장)

```text

# TRADE\_LIFECYCLE 정본 v0.1.5 (통합본)

\*\*(OPEN 1회 + CLOSE 1회 + 복기 / Indicator-agnostic / Deterministic / LLM 금지)\*\*

---

## 0) 목적과 목표

### 0-1. 목적

지표 엔진(FVG/OB/CHANNEL/TRENDLINE)이 생성한 \*\*OPEN 알림(REACTION/ENTRY\_WINDOW\_OPEN)\*\* 을 입력으로 받아,  
시스템이 \*\*운용 관점의 TP/SL/무효화/타임아웃을 결정론적으로 고정(Plan 동결)\*\* 하고,  
이후 가격 진행을 모니터링하여 \*\*CLOSE 알림(익절/손절/무효화/시간초과) 1회\*\*를 반드시 출력하며,  
CLOSE 시점에 \*\*규칙 기반 복기(익절: 잘한점 / 손절: 왜 손절 · 보완점)\*\* 를 함께 출력한다.

### 0-2. 핵심 목표(고정)

- \*\*OPEN 1회 + CLOSE 1회 = 1:1 완결\*\* (Orphan Plan 금지)
- \*\*결정론(재현성)\*\*: 동일 입력 캔들 스트림이면 출력 문자열이 1글자까지 동일
- \*\*롱/숏 대칭 지원\*\*
- \*\*LLM 기반 문구 생성 금지\*\* (복기/사유는 템플릿+규칙)
- \*\*HALT 구간 OPEN 금지\*\* (하드 방어 우선)

### 0-3. 비목적(Non-goals)

- 실제 체결/포지션 보유 여부는 필수 입력이 아니다.  
단, `entryFillPrice`가 제공되면 \*\*실체결 기준 R\*\*을 추가로 계산/출력할 수 있다(선택).

---

## 1) 용어/기본 정의

### 1-1. 캔들(bar)

- `open, high, low, close, openTime, closeTime`
- \*\*wick(심지)\*\* = `high/low` (극값)
- \*\*close(종가)\*\* = `close`
- 판정 규칙(고정):
  - \*\*HARD\_TP/HARD\_SL\*\*: wick 기반
  - \*\*SOFT\_INVALID\*\*: close 기반
  - 모든 time 기록은 bar의 `closeTime` 기준

### 1-2. Tick 정규화(결정론 + 실전 보수)

심볼마다 `tickSize` 존재. 모든 가격은 tick 정규화된 값으로만 저장/출력.

정규화 함수(고정):

- `roundToTick(x)` = tick 반올림
- `floorToTick(x)` = tick 내림
- `ceilToTick(x)` = tick 올림

방향별 보수 규칙(고정):

- **\*\*LONG\*\***
  - Stop(손절) = `floorToTick`
  - TP(익절) = `ceilToTick`
- **\*\*SHORT\*\***
  - Stop(손절) = `ceilToTick`
  - TP(익절) = `floorToTick`

### 1-3. bps

- `bps(diff, ref) = abs(diff) / ref \* 10000`
- ref는 기본적으로 `entryRefPrice`.

---

## 2) 결정론 LOCK(전역 고정)

### 2-1. time(UTC)

- 모든 time은 **\*\*UTC ISO-8601\*\***로 출력.
- `openTime` = OPEN을 발생시킨 bar의 closeTime
- `closeTime` = CLOSE가 확정된 bar의 closeTime

### 2-2. 배열 출력 정렬

- `tags[]`, `strengthCodes[]`, `weaknessCodes[]`는
  - **\*\*중복 제거(unique)\*\***
  - **\*\*lexicographic sort(사전순)\*\*** 후 출력(고정)

### 2-3. 텍스트 생성 금지

- 복기/사유는 **\*\*LLM/확률적 생성 금지\*\***
- 오직 코드→고정 템플릿 매핑으로만 생성(고정)

---

## 3) 입력 계약: OPEN\_INTENT (SEND\_OPEN Contract 내장)

본 정보는 아래 입력을 정확히 받았다고 가정한다. 누락 시 `INVALID\_INPUT` 또는 `POLICY\_MISSING`으로 처리(Plan 생성 금지).

### 3-1. 필수 필드(누락 금지)

- `symbol`
- `dir` = `LONG | SHORT`
- `eventType` = `REACTION | ENTRY\_WINDOW\_OPEN`

- `openTime` (UTC ISO, 신호 bar closeTime)
- `source` = `FVG | OB | CHANNEL | TRENDLINE`
- `poiTier` = `D1\_POI | H4\_CORE | H1\_SETUP | M30\_SETUP | OTHER`
- `poiId`
- `entryBoundaryPrice` (tick 정규화)
- `hardInvalidationPrice` (tick 정규화)
- `tags[]` (string array)
- `policySnapshot.decision` = `ALLOW | BLOCK`
- `policySnapshot.regimeState` = `NORMAL | CAUTION | TRANSITION | HALT`

### ### 3-2. 선택 필드(있으면 사용)

- `score?`
- `collabStrength?` = `NONE | WEAK | OK | STRONG`
- `entryFillPrice?` (tick 정규화 권장, 실체결가)

`policySnapshot` 선택 필드(있으면 사용):

- `policySnapshot.c\_bps?` (총비용 bps; fee+spread+slippage 보수 추정)
- `policySnapshot.sc?` (stop 대비 비용 스코어/비율 등 정책 결과)

---

## ## 4) TradePlan 데이터 모델(고정)

### ### 4-1. planKey / planId(고정)

- `planKey` = (symbol, dir, poiId)
- `planId` = symbol + "|" + dir + "|" + poiId + "@" + openTime

### ### 4-2. 상태(status, 고정)

- `OPEN | CLOSING | CLOSED`

### ### 4-3. TradePlan 필수 필드(OPEN 시점 동결)

- `planId, planKey`
- `symbol, dir, source, poiTier, poiId, eventType`
- `openTime`
- 가격(동결):
  - `entryRefPrice`
  - `entryBoundaryPrice`
  - `hardInvalidationPrice`
  - `stopPrice`
  - `tpPrice`
  - `tpMode` = LIQ | RR
  - `rrBase`
  - `rrChosen`

- `rrMaxUsed`
- 파라미터(동결):
  - `atrM5\_14\_atOpen`
  - `stopBuffer`
  - `entryQuality = IDEAL | VALID | LATE`
  - `timeoutMinutes`
  - `timeoutDueTime`
  - **\*\*tpLiq 관련 동결값\*\***
    - `tpLiqTf = H1 | M30`
    - `atrLiq\_14\_atOpen` ← **\*\*tpLiq용 ATR 동결\*\***
- 모니터링 누적(동결 아님, 진행 중 업데이트):
  - `mfeR` ← 최대 유리 진행
  - `maeR` ← 최대 불리 진행
- 종료 필드(종료 시 확정):
  - `closeTime`
  - `outcome = HARD\_TP | HARD\_SL | SOFT\_INVALID | TIMEOUT`
  - `exitPrice`
  - `bothHit` (true/false)
  - `timeoutSign = PROFIT | LOSS | FLAT | na`
  - `strengthCodes[]`
  - `weaknessCodes[]`
  - `rGross`, `rAfterCost`
  - `rFillGross?`, `rFillAfterCost?` (entryFillPrice 제공 시)

---

## 5) Plan 생성 규칙(OPEN 1:1 강제 + SUPPRESS)

### 5-1. 생성 트리거(고정)

- 시스템이 실제로 OPEN 알림을 **\*\*발송하기로 결정한 순간\*\***에만 TradePlan을 생성한다.
- 생성 시 `status = OPEN`.

### 5-2. 중복 OPEN 억제(고정)

- 동일 `planKey`에 대해 기존 plan이 `OPEN` 또는 `CLOSING`이면 **\*\*새 OPEN 생성/발송 금지(SUPPRESS\_DUPLICATE)\*\***

권장 로그(고정 템플릿):

```
- `[PLAN][SUPPRESS] time={openTime} reason=DUPLICATE symbol={symbol} dir={dir} poiId={poiId}`
```

### 5-3. (P0/P1) POLICY Guard(고정, 필수)

다음 중 하나라도 만족하면 **\*\*OPEN 발송 금지 + Plan 생성 금지\*\***:

(A) POLICY\_MISSING(고정)

```
- `policySnapshot`이 missing/null 이거나
- `policySnapshot.decision` 누락이거나
- `policySnapshot.regimeState` 누락이면
→ **SUPPRESS_POLICY_MISSING**
```

권장 로그(고정):

```
- `[PLAN][SUPPRESS] time={openTime} reason=POLICY_MISSING symbol={symbol} dir={dir} poiId={poiId}`
```

(B) HALT\_OR\_BLOCK(고정)

```
- `policySnapshot.decision == BLOCK`
- OR `policySnapshot.regimeState == HALT`
→ **SUPPRESS_HALT_OR_BLOCK**
```

권장 로그(고정):

```
- `[PLAN][SUPPRESS] time={openTime} reason=HALT_OR_BLOCK symbol={symbol} dir={dir} poiId={poiId}`
```

> HALT는 “ 약점 코드 ” 가 아니라 \*\*거래 금지\*\*이므로 Plan을 만들면 안 된다(고정).

### 5-4. (P1) LATE 조건부 SUPPRESS(고정)

먼저 `entryQuality`를 계산한 뒤, 아래 조건이면 \*\*OPEN/Plan SUPPRESS\*\* 한다.

SUPPRESS 조건(고정):

```
- `entryQuality == LATE`
- AND `collabStrength ∈ {na, NONE, WEAK}`
- AND `poiTier ∈ {H1_SETUP, M30_SETUP, OTHER}`
```

예외(고정): 아래 중 하나면 LATE라도 허용

```
- `poiTier ∈ {D1_POI, H4_CORE}`
- OR `collabStrength ∈ {OK, STRONG}`
```

권장 로그(고정):

```
- `[PLAN][SUPPRESS] time={openTime} reason=LATE_LOW_CONV symbol={symbol} dir={dir} poiId={poiId}`
...

```

## BLOCK 2/3 — TRADE\_LIFECYCLE 정본 v0.1.5 (통합본) (6~10장)

```text

6) 가격 계산 LOCK(OPEN 시점 1회 계산 후 동결)

6-1. entryRefPrice(고정)

```
- `entryRefPrice = roundToTick(signalBar.close)`
```

6-2. 모니터 ATR 동결(M5, 고정)

- 모니터링 TF = **M5**
- `atrM5\_14\_atOpen = ATR(M5,14) at openTime`

6-3. stopBuffer(고정)

상수(고정):

- `STOP\_BUFFER\_ATR = 0.15`
- `STOP\_BUFFER\_MIN\_TICKS = 2`

정의:

- `stopBufferRaw = max(STOP\_BUFFER\_MIN\_TICKS\*tickSize, atrM5\_14\_atOpen\*STOP\_BUFFER\_ATR)`
- `stopBuffer = roundToTick(stopBufferRaw)`

6-4. stopPrice(핵심 LOCK, 고정)

- LONG:
 - `stopPriceRaw = hardInvalidationPrice - stopBuffer`
 - `stopPrice = floorToTick(stopPriceRaw)`
- SHORT:
 - `stopPriceRaw = hardInvalidationPrice + stopBuffer`
 - `stopPrice = ceilToTick(stopPriceRaw)`

안전 보정(고정):

- LONG: `stopPrice <= entryRefPrice - 1\*tickSize` 아니면 `stopPrice = entryRefPrice - 1\*tickSize`
- SHORT: `stopPrice >= entryRefPrice + 1\*tickSize` 아니면 `stopPrice = entryRefPrice + 1\*tickSize`

> (채택 이유) CHANNEL/TL처럼 boundary==invalidation이어도 **s>0** 보장 → 비용/SC/RR 붕괴 방지.

6-5. entryQuality(고정)

- `distToBoundary = abs(entryRefPrice - entryBoundaryPrice)`
- 임계(고정):
 - `ENTRY\_Q\_IDEAL\_ATR = 0.05`
 - `ENTRY\_Q\_VALID\_ATR = 0.10`

판정:

- IDEAL: `distToBoundary <= atrM5\_14\_atOpen \* 0.05`
- VALID: `distToBoundary <= atrM5\_14\_atOpen \* 0.10`
- else: LATE

6-6) TP 계산(유동적 값 + 고정 규칙)

6-6-1. RR 가드레일(고정)

- `RR\_MIN = 1.20`

RR_MAX는 tier별로 고정(채택):

- `RR\_MAX\_DEFAULT = 3.00`

- `RR\_MAX\_H4\_CORE = 3.50`

- `RR\_MAX\_D1\_POI = 4.00`

- `RR\_MAX\_D1\_POI\_STRONG = 4.50`

`rrMaxUsed` 선택 규칙(고정):

- if `poiTier==D1\_POI` and `collabStrength==STRONG` → `rrMaxUsed=4.50`

- else if `poiTier==D1\_POI` → `rrMaxUsed=4.00`

- else if `poiTier==H4\_CORE` → `rrMaxUsed=3.50`

- else → `rrMaxUsed=3.00`

6-6-2. RR_BASE(고정, entryQuality 기반)

- IDEAL: `RR\_BASE = 1.80`

- VALID: `RR\_BASE = 1.50`

- LATE: `RR\_BASE = 1.30`

- `rrBase = RR\_BASE` (OPEN 로그/감사 추적용, 고정)

6-6-3. stopDistance s(고정)

- `s = abs(entryRefPrice - stopPrice)`

- `s <= 0`이면 입력/정규화 오류(정상 경로에서 발생 금지)

6-6-4. tpRR(고정)

- `rrChosen = clamp(rrBase, RR\_MIN, rrMaxUsed)`

- LONG:

- `tpRR\_raw = entryRefPrice + rrChosen \* s`

- `tpRR = ceilToTick(tpRR\_raw)`

- SHORT:

- `tpRR\_raw = entryRefPrice - rrChosen \* s`

- `tpRR = floorToTick(tpRR\_raw)`

6-6-5. tpLiq(유동성 타겟, 결정론 동결 포함)

tpLiq는 “있으면 사용”, 없으면 tpRR로 fallback(고정).

(A) tpLiqTf 매핑(고정)

- `poiTier ∈ {D1\_POI, H4\_CORE, H1\_SETUP}` → tpLiqTf = H1`

- `poiTier == M30\_SETUP` → tpLiqTf = M30`

- else -> H1

(B) atrLiq_14_atOpen 동결(고정)

- `atrLiq\_14\_atOpen = ATR(tpLiqTf,14) at openTime`
- EQ 판단/필터는 **반드시 이 동결값**만 사용(실시간 ATR 금지)

(C) pivot lookback(고정)

- `H1\_TP\_LOOKBACK\_BARS = 300`
- `M30\_TP\_LOOKBACK\_BARS = 500`

(D) pivot/EQ 정의(고정)

- pivot은 fractal pivotLen=3 확정 pivot 사용
- pivot 확정: pivot 기준봉 p의 `p+3 closeTime` 시점에 확정

사용 가능 pivot 조건(룩어헤드 금지, 고정):

- pivot 기준봉 p에 대해 `pivotConfirmTime = bar\_tf[p+3].closeTime`
- `pivotConfirmTime < openTime` 인 pivot만 tpLiq 후보로 인정한다
- 이를 만족하지 않는 pivot은 tpLiq 후보에서 제외한다

EQ 밴드(고정):

- `EQ\_BAND\_ATR = 0.10`
- 연속 pivot 2개가 같은 타입일 때
 - `abs(p2 - p1) <= atrLiq\_14\_atOpen \* 0.10` → EQH/EQL

(E) 후보 레벨/선택(고정)

- 후보 우선순위:
 - 1) EQH/EQL
 - 2) 없으면 pivot swing high/low

선택:

- LONG: `entryRefPrice` 위 후보 중 **가장 가까운 레벨**
- SHORT: `entryRefPrice` 아래 후보 중 **가장 가까운 레벨**
- 없으면 `tpLiq = null`

(F) tpLiq 적용 여부(고정)

- `tpLiq == null` → tpMode=RR, tpPrice=tpRR
- else:
 - `rrLiq = abs(tpLiq - entryRefPrice) / s`
 - `RR\_MIN <= rrLiq <= rrMaxUsed` 이면:
 - tpMode=LIQ
 - tpPrice:
 - LONG: `tpPrice = ceilToTick(tpLiq)`

- SHORT: `tpPrice = floorToTick(tpLiq)`
- 아니면 tpMode=RR, tpPrice=tpRR

7) timeoutMinutes(고정 + “ 초기값 ” 명시)

기본값(고정):

- `TIMEOUT\_D1\_POI\_MIN = 360`
- `TIMEOUT\_H4\_CORE\_MIN = 240`
- `TIMEOUT\_H1\_SETUP\_MIN = 120`
- `TIMEOUT\_M30\_SETUP\_MIN = 90`
- `TIMEOUT\_OTHER\_MIN = 120`

매핑(고정):

- poiTier별로 해당 값을 사용

변경 규칙(고정):

- timeout 값 변경은 **런타임 적응 금지(v0.1.x)**
- 변경은 반드시 **버전업/패치로만** 수행한다.

8) 모니터링 및 CLOSE 판정(M5 고정)

8-1. 평가 시점(고정)

- `onBarClose(M5)`에서만 평가.

****누적/판정 시작 bar(고정, P0):****

- `bar.closeTime > openTime` 인 M5 bar부터만
 - MFE/MAE 누적
 - HARD_TP/HARD_SL 히트 판정
 - SOFT_INVALID 판정
 - TIMEOUT 판정

을 수행한다.

- `bar.closeTime <= openTime` 인 bar는 모든 모니터링/판정에서 제외한다(openTime bar 포함).

8-2. MFE/MAE 누적(고정)

각 평가 대상 M5 bar에서 갱신:

- `favR`(유리 진행, wick):
 - LONG: `(bar.high - entryRefPrice) / s`
 - SHORT: `(entryRefPrice - bar.low) / s`

```
- `advR`(불리 진행, wick):  
- LONG: `(entryRefPrice - bar.low) / s`  
- SHORT: `(bar.high - entryRefPrice) / s`
```

갱신(고정):

```
- `mfeR = max(prev_mfeR, favR)`  
- `maeR = max(prev_maeR, advR)`
```

8-3. HARD_TP / HARD_SL(고정, wick)

```
- LONG:  
- `slHit = (bar.low <= stopPrice)`  
- `tpHit = (bar.high >= tpPrice)`  
- SHORT:  
- `slHit = (bar.high >= stopPrice)`  
- `tpHit = (bar.low <= tpPrice)`
```

```
`bothHit = (slHit && tpHit)`
```

8-4. SOFT_INVALID(고정, close)

조건 A(필수):

```
- LONG: `bar.close <= hardInvalidationPrice - 1*tickSize`  
- SHORT: `bar.close >= hardInvalidationPrice + 1*tickSize`
```

조건 B(선택, 구현 가능 시):

```
- 동일 poiId에 대해 지표 엔진이 INVALID(endTime)을 발생시킨 경우,  
- `invalidTime = endTime`  
- SOFT_INVALID는 **first M5 bar with (closeTime > openTime) AND (closeTime >= invalidTime)** 에서 발생한 것으로 간주(고정)
```

8-5. TIMEOUT(고정)

```
- `bar.closeTime >= timeoutDueTime` 인 최초 평가 대상 bar에서 TIMEOUT
```

8-6. 종료 우선순위(LOCK, 고정)

아래 순서로 1개만 선택:

- 1) HARD_SL
- 2) HARD_TP
- 3) SOFT_INVALID
- 4) TIMEOUT

동일 bar에서 bothHit이면 **SL 우선(보수적, 고정)**.

8-7. closeTime / exitPrice(고정)

```
- `closeTime = 해당 종료를 확인한 M5 bar.closeTime`
```

```
`exitPrice`:
- HARD_SL → `stopPrice`
- HARD_TP → `tpPrice`
- SOFT_INVALID → `roundToTick(bar.close)`
- TIMEOUT → `roundToTick(bar.close)`
```

HARD_SL exitPrice 주석(고정):

- HARD_SL의 `exitPrice = stopPrice`는 바 단위 모델에서의 **이론적 체결(트리거) 가격**이다.
- 실제 체결은 슬리피지로 더 불리할 수 있으며, 그 보정은 `policySnapshot.c\_bps` 또는 `rFill\*`(실체결 입력)에서만 다룬다.

9) timeoutSign(고정)

```
- outcome != TIMEOUT → `timeoutSign = na`
- outcome == TIMEOUT이면 `rGross` 계산 후:
  - `rGross > +0.10` → `PROFIT`
  - `rGross < -0.10` → `LOSS`
  - else → `FLAT`
```

10) 복기(Review) 생성 규칙(LLM 금지)

10-1. R 계산(고정)

```
- `s = abs(entryRefPrice - stopPrice)`
- `rGross = dirSign * (exitPrice - entryRefPrice) / s`
  - LONG: `dirSign=+1`
  - SHORT: `dirSign=-1`
```

비용 반영(선택):

```
- `policySnapshot.c_bps` 있으면
  - `costPrice = entryRefPrice * (c_bps/10000)`
  - `costR = costPrice / s`
  - `rAfterCost = rGross - costR`
- 없으면 `rAfterCost = na`
```

10-2. 실체결 R(선택)

`entryFillPrice` 제공 시:

```
- `sFill = abs(entryFillPrice - stopPrice)`
- `sFill <= 0`이면 `sFill = abs(entryRefPrice - stopPrice)`로 대체(고정)
- `rFillGross = dirSign * (exitPrice - entryFillPrice) / sFill`
- 비용 반영은 동일하게 `rFillAfterCost` 계산 가능
```

- 값이 없으면 `na`
``

BLOCK 3/3 — TRADE_LIFECYCLE 정본 v0.1.5 (통합본) (11~15장)

```text

## 11) Strength/Weakness 코드(고정 목록)

### 11-1. StrengthCodes(예시, 고정)

- `S\_POI\_TIER\_HIGH` : poiTier  $\in$  {D1\_POI, H4\_CORE}
- `S\_COLLAB\_STRONG` : collabStrength == STRONG
- `S\_ENTRY\_IDEAL` : entryQuality == IDEAL
- `S\_TP\_MODE\_LIQ` : tpMode == LIQ
- `S\_SCORE\_HIGH` : score exists AND score  $\geq$  90
- `S\_POLICY\_OK` : policySnapshot.regimeState == NORMAL (있을 때만)

### 11-2. WeaknessCodes(예시, 고정)

- `W\_POLICY\_CAUTION` : regimeState  $\in$  {CAUTION, TRANSITION} (있을 때만)
- `W\_ENTRY\_LATE` : entryQuality == LATE
- `W\_TP\_MODE\_RR` : tpMode == RR
- `W\_RR\_LOW` : rrChosen  $<$  1.40
- `W\_SC\_LOW` : policySnapshot.sc exists AND sc  $<$  3.5
- `W\_BOTH\_HIT` : bothHit == true
- `W\_GAVE\_BACK\_PROFIT` :  
((outcome  $\in$  {HARD\_SL, SOFT\_INVALID}) AND (mfeR  $\geq$  1.00))  
OR ((outcome == TIMEOUT) AND (timeoutSign == LOSS) AND (mfeR  $\geq$  1.00))

### 11-3. 출력 정렬(고정)

- 모든 codes는 조건 만족 시 포함
- 출력은 unique + lexicographic sort

---

## 12) 복기 메시지(고정 템플릿) 생성 규칙

문장 생성이 아니라 \*\*코드→고정 문구 매핑\*\*만 허용.

### 12-1. Root 문구(고정)

- HARD\_TP: “익절(TP) 체결”
- HARD\_SL: “하드 스탑(SL) 체결”
- SOFT\_INVALID: “종가 기준 무효화(아이디어 붕괴)”
- TIMEOUT: “타임아웃(기대 진행 부재)”

TIMEOUT 보조 문구(고정):

- timeoutSign=PROFIT: “ 수익 구간에서 시간 종료 ”
- timeoutSign=LOSS: “ 손실 구간에서 반등 실패로 시간 종료 ”
- timeoutSign=FLAT: “ 방향성 부족으로 시간 종료 ”

### 12-2. 잘한점/부족한점 선택 규칙(고정)

- outcome == HARD\_TP:
  - “ 잘한점 ” = StrengthCodes 중 \*\*우선순위 상위 2개\*\*
  - 우선순위(고정):
    - `S\_COLLAB\_STRONG > S\_ENTRY\_IDEAL > S\_TP\_MODE\_LIQ > S\_POI\_TIER\_HIGH > S\_POLICY\_OK > S\_SCORE\_HIGH`
- outcome ∈ {HARD\_SL, SOFT\_INVALID}:
  - “ 왜 종료 ” = Root 문구
  - “ 보완 ” = WeaknessCodes 중 우선순위 상위 2개
  - 단, outcome==HARD\_SL일 때 Weakness 우선순위(고정):
    - `W\_RR\_LOW > W\_GAVE\_BACK\_PROFIT > W\_ENTRY\_LATE > W\_POLICY\_CAUTION > W\_TP\_MODE\_RR > W\_SC\_LOW > W\_BOTH\_HIT`
- outcome == TIMEOUT:
  - “ 왜 종료 ” = Root + timeoutSign 보조 문구
  - “ 보완 ” = WeaknessCodes 상위 2개(우선순위는 SL 규칙과 동일)

---

## 13) 출력 포맷(로그/텔레그램) (고정)

### 13-1. OPEN(1회)

단일 라인(필드 순서 고정):

```
`[PLAN][OPEN] time={openTime} planId={planId} symbol={symbol} dir={dir} source={source} poiTier={poiTier} poiId={poiId} eventType={eventType}
entryRef={entryRefPrice} boundary={entryBoundaryPrice} invalid={hardInvalidationPrice} stop={stopPrice} tp={tpPrice} tpMode={LIQ|RR} rrBase={rrBase}
rr={rrChosen} rrMax={rrMaxUsed} entryQ={IDEAL|VALID|LATE} timeoutMin={timeoutMinutes} tags={t1|t2|...} score={na|x} collab={na|NONE|WEAK|OK|STRONG}
regime={NORMAL|CAUTION|TRANSITION|HALT} sc={na|x} c_bps={na|x}`
```

### 13-2. CLOSE(1회)

```
`[PLAN][CLOSE] time={closeTime} planId={planId} symbol={symbol} dir={dir} outcome={HARD_TP|HARD_SL|SOFT_INVALID|TIMEOUT} exit={exitPrice}
bothHit={true|false} timeoutSign={PROFIT|LOSS|FLAT|na} mfeR={x.xx} maeR={x.xx} openTime={openTime} closeTime={closeTime} rGross={x.xx} rAfterCost={na|x.xx}
rFillGross={na|x.xx} rFillAfterCost={na|x.xx} strengths={S1|S2|...} weaknesses={W1|W2|...}`
```

### 13-3. SUPPRESS(권장, 고정 템플릿)

```
`[PLAN][SUPPRESS] time={openTime} reason={DUPLICATE|POLICY_MISSING|HALT_OR_BLOCK|LATE_LOW_CONV|INVALID_INPUT} symbol={symbol} dir={dir} poiId={poiId}`
```

---

## 14) 완결성(Orphan 방지, 고정)

- 모든 plan은 반드시 1회 종료(HARD\_TP/HARD\_SL/SOFT\_INVALID/TIMEOUT)해야 한다.

- TIMEOUT이 존재하므로 Orphan Plan은 논리적으로 불가능해야 한다(고정).

---

## 15) 승격 Gate(PASS/FAIL, 고정)

PASS 조건:

- 1) 동일 리플레이 입력 2회 실행 시 OPEN/CLOSE 문자열이 1글자도 다르지 않음
- 2) 모든 OPEN plan에 대해 CLOSE가 정확히 1회 존재(Orphan=0)
- 3) bothHit 케이스에서 항상 SL로 귀결되고 `bothHit=true`가 기록됨
- 4) tpLiq 관련 ATR은 반드시 `atrLiq\_14\_atOpen`만 사용(실시간 ATR 사용 금지)
- 5) tpLiq pivot은 반드시 `pivotConfirmTime < openTime` 조건을 만족하는 것만 사용(룩어헤드 금지)
- 6) 모니터링/판정/누적은 반드시 `bar.closeTime > openTime` 인 M5 bar부터 시작(openTime bar 포함 이전 bar 제외)
- 7) policySnapshot missing/null 또는 decision/regime 누락이면 OPEN/Plan 생성이 발생하지 않음(POLICY\_MISSING SUPPRESS 로그로만 남김)
- 8) policySnapshot이 HALT/BLOCK이면 OPEN/Plan 생성이 발생하지 않음(HALT\_OR\_BLOCK SUPPRESS 로그로만 남김)

```

16) PATCH ADDENDUM (v0.1.5 유지 + "추가 LOCK"만 반영)

- 본 섹션은 기존 본문(Blocks 1/2/3)의 어떤 문장도 수정/삭제하지 않는다.
- 아래 항목은 "추가"로만 적용되는 공식 LOCK 이다.

16-1) Price Source LOCK (BTC Futures)

- TradeLifecycle의 모든 판정(ENTRY quality, MFE/MAE, HARD_TP/HARD_SL, SOFT_INVALID, TIMEOUT)은 모니터링 입력 bar(M5)의 OHLC 기준으로만 계산한다.
- bar(M5)의 OHLC는 Binance USD-M Futures 표준 kline(= LAST trade price) 기반으로 고정한다.
(Mark price 기반 평가는 v0.2+ 확장 범위; 본 버전에서는 금지)

16-2) Data Gap / Backfill Guard (Determinism)

- TradeLifecycle은 M5 closeTime이 5분 간격으로 "연속" 입력되는 것을 전제로 한다.
- 아래 중 하나라도 감지되면 해당 구간은 DATA_GAP 으로 간주한다(고정):
 - (A) `curr.closeTime - prev.closeTime > 5m`
 - (B) closeTime 역전(out-of-order) 발생
- DATA_GAP 발생 시 행동(고정):
 - (1) 새 Plan 생성(SEND_OPEN → OPEN) 금지: reason=DATA_GAP 로 SUPPRESS 처리
 - (2) 기존 OPEN Plan의 CLOSE 판정도 "보류"한다
(누락 bar 때문에 TP/SL/INVALID hit 판정이 뒤틀릴 수 있음)
 - (3) 런타임이 REST backfill로 누락 bar를 재주입하여 closeTime 연속성이 복구된 후에만 재개한다

16-3) Zone-level Dedup Key (Cross-indicator overlap spam 방지)

- 기존 planKey(= symbol|dir|poiId) 외에 zoneKey를 추가한다.
 - OPEN_INTENT에 아래 필드가 제공되면 zoneKey는 이를 우선 사용한다:
 - * poiZoneBottom (tick normalized)
 - * poiZoneTop (tick normalized)
 - 위 필드가 없으면 zoneKey는 다음 fallback으로 계산한다(고정):
 - zoneBottom = min(entryBoundaryPrice, hardInvalidationPrice)
 - zoneTop = max(entryBoundaryPrice, hardInvalidationPrice)
 - zoneKey = symbol|dir|tick(zoneBottom)~tick(zoneTop)
 - DUPLICATE 정의(고정):
동일 symbol, 동일 dir에서 "활성 Plan(OPEN 또는 CLOSING)"이 zoneKey 기준으로 이미 존재하면 새 OPEN은 SUPPRESS 한다. reason=DEDUP_ZONE
(planKey가 달라도 zoneKey가 같으면 중복으로 간주)
-